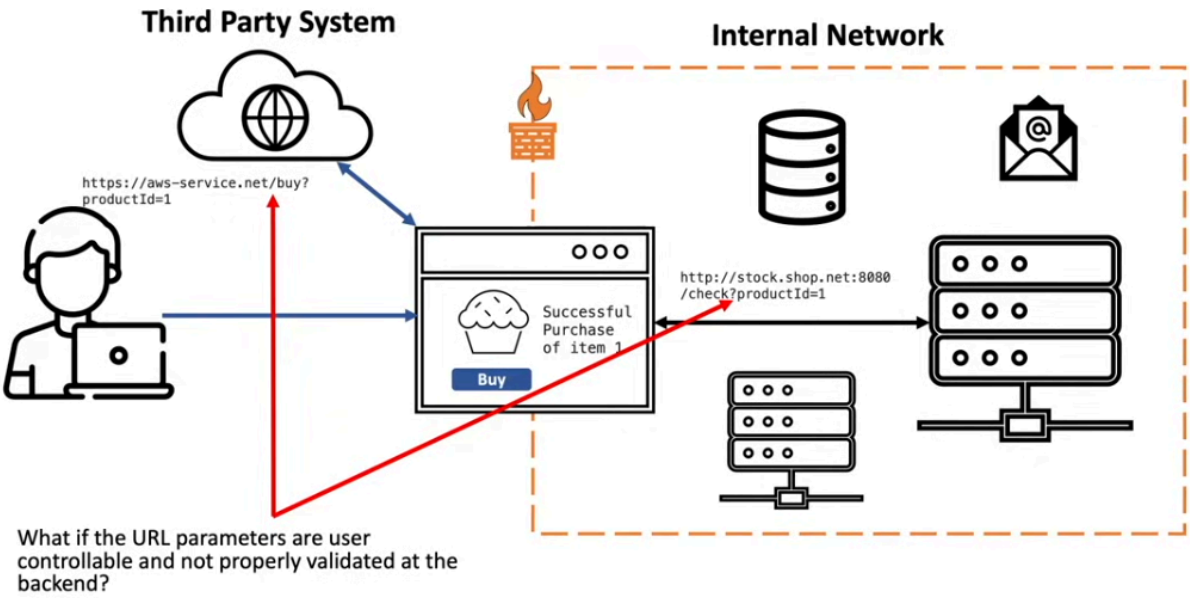
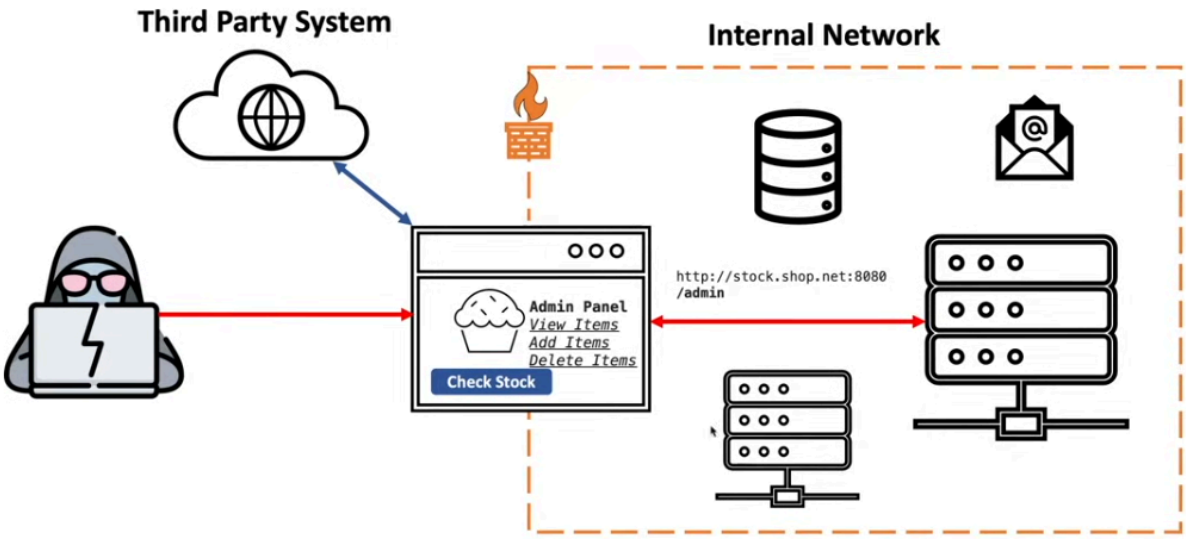
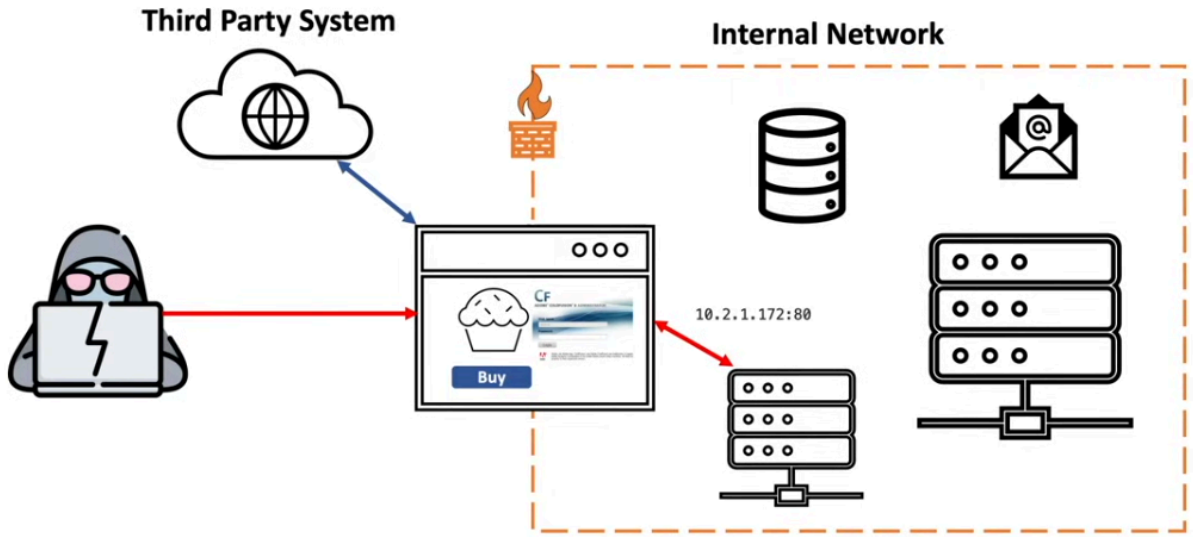
SERVER-SIDE REQUEST FORGERY (SSRF)



Example 1:



Example 2:



- SSRF is a vulnerability class that occurs when an application is fetching a remote resource without first validating the user supplied URL.

TYPES OF SSRF

- Regular / In Band
- Blind / Out of Brand

IMPACT OF SSRF ATTACKS

- Can lead to sensitive information disclosure, scan of internal network compromise of internal services, remote code execution, etc.

HOW TO FIND SSRF VULNERABILITIES

- Map the application
  - Identify any request parameters that contain hostnames, IP addresses or full URLs
- For each request parameter, modify its value to specify an alternative resource and observe how the application responds
  - If a defense is in place, attempt to circumvent it using known techniques

- For each request parameter, modify its value to a server on the internet that you control and monitor the server for incoming requests
    - If no incoming connections are received, monitor the time taken for the application to respond

---

HOW TO EXPLOIT SSRF VULNERABILITIES

EXPLOITING REGULAR / IN-BAND SSRF

Request:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-
form-urlencoded
Content-Length: 118


stockApi=http://stock.weliketoshop
.net:8080/product/stock/check%3Fpr
oductId%3D6%26storeId%3D1
```

Response:

```
HTTP/1.1 200 OK
Content-Type: text/plain;
charset=utf-8
Connection: close
Content-Length: 3



506
```

Request:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-
form-urlencoded
Content-Length: 118




stockApi=http://localhost/admin
```

Response:

- If the application allows for user-supplied arbitrary UTLs, try the following attacks:
    - Determine if a port number can be specified
    - If successful, attempt to port-scan the internal network using Burp Intruder
    - Attempt to connect to other services on the loopback address
- If the application does not allow for arbitrary user-supplied URLs, try to bypass defenses using the following techniques:
    - Use different encoding schemes
        - Decimal-encoded version of 127.0.0.1 is 2130706433
        - 127.1 resolves to 127.0.0.1
        - Octal representation of 127.0.0.1 is 017700000001

    - Register a domain name that resolves to internal IP address (DNS Rebinding)
    - Use your own server that redirects to an internal IP address (HTTP Redirection)
    - Exploit inconsistencies in URL parsing

EXPLOITING BLIND / OUT-OF-BAND SSRF

- If the application is vulnerable to blind SSRF, try to exploit the vulnerability using the following techniques:
    - Attempt to trigger an HTTP request to an external system you control and monitor the system for network interactions from the vulnerable server
        - Can be done using Burp Collaborator
    - In defenses are put in place, use the techniques mentioned in previously to obfuscate the external malicious domain
    - Once youve proved that the application is vulnerable to blind SSRF, you need to detrmine what your end goal is
        - An example would be to prove for other vulnerabilities on the server itself or other backend systems

---

PREVENTING SSRF VULNERABILITIES

Defense in depth approach:

- Application layer defenses
- Network layer defenses

APPLICATION LAYER DEFENSES

- Sanitize and validate all client supplied input data
- Enforce the URL schema, port , and destination with a positive allow list
- Do not send raw responses to clients
- Disable HTTP redirections

Note: Do not mitigate SSRF vulnerabilities using deny lists or regular expressions.

NETWORK LAYER DEFENSES

- Segment remote resource access functionality in separate networks to reduce the impact of SSRF
- Enforce "deny by default" firewall policies or network access control rules to block all but essential intranet traffic